

```

import os
import re
import pandas as pd

def clean_tweet(text):
    """
    Cleans the tweet by COMPLETELY REMOVING usernames.
    It also removes leading/trailing quotes and extra white spaces.
    """
    # Remove leading and trailing quotation marks
    text = text.strip('"\'')

    # Replace any username with nothing
    text = re.sub(r'@\w+', '', text)

    # Remove any extra spaces
    text = " ".join(text.split())

    return text

def load_and_preprocess_data(base_directory):
    """
    Iterates through the folder structure, reads files, cleans text,
    and returns a combined Pandas DataFrame with guaranteed
    classifications.
    """
    data = []

    folders = ['airlinetweets/negative', 'airlinetweets/neutral',
               'airlinetweets/positive']

    print("Starting preprocessing...")

    for label in folders:
        folder_path = os.path.join(base_directory, label)

        if not os.path.exists(folder_path):
            print(f"Warning: Folder '{label}' not found in {base_directory}")
            continue

        # Iterate through every text file in the folder
        file_list = os.listdir(folder_path)

        for filename in file_list:
            if filename.endswith(".txt"):
                file_path = os.path.join(folder_path, filename)

                # Open and read the text file

```

```

        with open(file_path, 'r', encoding='utf-8') as file:
            tweet_text = file.read()

            # Clean the tweet (removes usernames)
            cleaned_text = clean_tweet(tweet_text)

            # Append to dataset and add the classification
            label

            data.append({
                'text': cleaned_text,
                'sentiment': label # This guarantees it is
classified!
            })

    df = pd.DataFrame(data)
    return df

def main():
    base_folder_path = '.'

    # Create the dataset
    training_df = load_and_preprocess_data(base_folder_path)

    print(f"Total tweets: {len(training_df)}")

    print(training_df['sentiment'].value_counts())

    # Save data in a CSV file
    output_filename = "cleaned_training_data.csv"
    training_df.to_csv(output_filename, index=False)

if __name__ == "__main__":
    main()

```

Starting preprocessing...

Total tweets: 4755

sentiment

airlinetweets/negative	1750
------------------------	------

airlinetweets/neutral	1515
-----------------------	------

airlinetweets/positive	1490
------------------------	------

Name: count, dtype: int64

```
import pandas as pd
```

```
import torch
```

```
from transformers import pipeline
```

```
from sklearn.metrics import classification_report
```

```
sent_topic_test = pd.read_csv('Sentiment-topic-test (1).tsv', sep='\t')
```

```
roberta_name = "cardiffnlp/twitter-roberta-base-sentiment-latest"  
sentiment_pipeline = pipeline("sentiment-analysis",  
model=roberta_name, tokenizer=roberta_name)
```

#Make the predictions and save them to a new column

```
def get_roberta_sentiment(text):  
    return sentiment_pipeline(text)[0]['label'].lower()
```

```
sent_topic_test['RoBERTa_Pred'] =  
sent_topic_test['text'].apply(get_roberta_sentiment)
```

```
print("\nRoBERTa Sentiment Metrics")  
print(classification_report(sent_topic_test['sentiment'],  
sent_topic_test['RoBERTa_Pred'], zero_division=0))
```

Loading weights: 100%|██████████| 201/201 [00:00<00:00, 7725.31it/s]

[transformers] RobertaForSequenceClassification LOAD REPORT from:
cardiffnlp/twitter-roberta-base-sentiment-latest

Key	Status		
roberta.pooler.dense.weight	UNEXPECTED		
roberta.pooler.dense.bias	UNEXPECTED		

Notes:

- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch.

RoBERTa Sentiment Metrics

	precision	recall	f1-score	support
negative	1.00	1.00	1.00	3
neutral	1.00	1.00	1.00	3
positive	1.00	1.00	1.00	4
accuracy			1.00	10
macro avg	1.00	1.00	1.00	10
weighted avg	1.00	1.00	1.00	10